

MISKOLCI EGYETEM

Gépészmérnöki és Informatikai Kar

Informatikai Tanszék

NYÁRI SZAKMAI GYAKORLAT BESZÁMOLÓ

Tanszéki Eszköznyilvántartó Webes Rendszer Tervezése, Megvalósítása és Infrastrukturális Integrációja

*Egy modern, háromrétegű, konténerizált informatikai leltár- és munkafolyamat-kezelő
webalkalmazás fejlesztése Spring Boot 3.3, React 18 és PostgreSQL 16 alapokon*

Készítette:

Bucskó Gábor

Mérnök-informatikus BSc

Neptun-kód: **PSD1T8**

Gyakorlati hely:

Miskolci Egyetem

Informatikai Tanszék

Szakmai gyakorlat (2026)

Miskolc, 2026. augusztus

Tartalomjegyzék

1. Fejezet — Webes nyilvántartó rendszerek tanulmányozása	3
1.1. Bevezetés a tanszéki eszökgazdálkodás kihívásaiba	3
1.2. Webes technológia-csoportok összehasonlító elemzése	3
1.2.1. PHP + MySQL alapú keretrendszerek (Laravel, Symfony)	
1.2.2. Java + Spring Boot + PostgreSQL ökoszisztéma	
1.2.3. Node.js + Express / NestJS + MongoDB architektúra	
1.2.4. Python + Django + PostgreSQL technológia	
1.3. Részletes összehasonlító értékelés és szempontrendszer	5
1.4. A választott technológiai stack indoklása	6
2. Fejezet — A rendszer tervezése	8
2.1. Részletes követelményanalízis és use-case modellezés	8
2.1.1. Funkcionális követelmények	
2.1.2. Nem-funkcionális követelmények	
2.2. Architektúra tervezés és rétegzett struktúra	9
2.3. Relációs adatmodell tervezés és séma-specifikáció	9
2.4. Biztonsági architektúra és adatvédelmi mechanizmusok	11
2.5. Rendszerszintű állapotgépek és folyamatmodellek	11
3. Fejezet — Alapvető panelek elkészítése és tesztelés	13
3.1. Felhasználói felület és komponens-panelek implementációja	13
3.1.1. Hitelesítési és jogosultságkezelő panelek	
3.1.2. Irányítópult (Dashboard) és áttekintő felület	
3.1.3. Eszköznyilvántartó és részletező panel	
3.1.4. Hierarchikus helyszínkezelő panel	
3.1.5. Jóváhagyási várólista és munkafolyamat-kezelő	
3.1.6. Felhasználó- és szerepkör-adminisztráció	
3.1.7. Audit napló és automatizált állapot-visszaállítás (Rollback)	
3.1.8. Szoftverlicenc-kezelő és tömeges adatimport panel	
3.2. Backend API és üzleti logikai réteg megvalósítása	15
3.3. Tesztelési stratégia és eredmények	16
4. Fejezet — Fejlesztési környezet és működés dokumentálása	18
4.1. Fejlesztési eszközkészlet és futtatókörnyezet	18
4.2. Monorepo projektstruktúra és kódszervezés	18
4.3. Alapvető panelek és munkafolyamatok működési leírása	19
4.4. Minőségbiztosítási folyamatok és CI/CD automatizáció	20
5. Fejezet — Tanszéki infrastruktúrába integrálás	21
5.1. Rendszer üzembe helyezési folyamat és inicializálás	21
5.2. Konténerizált többkomponensű Docker architektúra	21
5.3. Biztonsági megerősítés (Production Hardening)	22
5.4. Biztonsági mentés, adatvédelem és katasztrófa-helyreállítás	23
5.5. Titkosítási kulcsok és nulla állásidejű JWT rotáció	23
5.6. Rendszermonitorozás, egészség-ellenőrzés és strukturált naplózás	23
5.7. Hálózati integráció és Nginx reverse proxy	24
Összefoglalás és jövőbeli fejlesztési lehetőségek	25
Irodalomjegyzék	26

1. Fejezet — Webes nyilvántartó rendszerek tanulmányozása

1.1. Bevezetés a tanszéki eszközgazdálkodás kihívásaiba

Az egyetemi tanszékek mindennapi működésének elengedhetetlen feltétele a rendelkezésre álló anyagi és technológiai erőforrások pontos, naprakész és átlátható nyilvántartása. Az oktatási és kutatási feladatok ellátása során a tanszékek nagyszámú hardvereszközzel gazdálkodnak: asztali munkaállomások, nagy számítási kapacitású szerverek, hordozható számítógépek (laptopok), monitorok, hálózati berendezések (switchek, routerek), multimédiás eszközök (projektorok, digitális táblák), valamint speciális laboratóriumi mérőműszerek és fejlesztői kiegészítők alkotják az eszközparkot. Ezen felül a hardvereken futó szoftverek licenceinek, verzióinak és érvényességi idejének nyomon követése is kiemelt adminisztrációs terhet jelent.

Hagyományosan sok felsőoktatási intézményben az eszközök nyilvántartása manuális eszközökkel — leggyakrabban táblázatkezelő fájlokban (pl. Microsoft Excel vagy LibreOffice Calc) — vagy elavult, szigetszerűen működő asztali alkalmazások segítségével történik. Ez a megközelítés azonban számos kritikus problémát vet fel:

- **Adatkonzisztencia és párhuzamossági problémák:** Több oktató és adminisztrátor egyidejű munkavégzése esetén a fájlalapú nyilvántartások gyorsan szétválnak, egymásnak ellentmondó verziók keletkeznek, ami a fizikai leltár és a nyilvántartás elcsúszásához vezet.
- **Felelősségi körök és jóváhagyási láncok hiánya:** A táblázatokban nem kényszeríthető ki szigorú jogosultságkezelés. Bárki akaratlanul átírhat adatokat anélkül, hogy az eszközfelelős oktató vagy a tanszékvezető jóváhagyása megtörtént volna.
- **Auditálhatóság és nyomonkövethetőség hiánya:** Nincs strukturált naplózás arra vonatkozóan, hogy egy adott eszközt ki, mikor, melyik teremből melyik irodába helyezték át, ki végzett rajta karbantartást, vagy mikor és miért selejtezték le.
- **Csatolt dokumentumok kezelésének nehézsége:** Az eszközökhöz kapcsolódó számlák, jótállási jegyek, átadás-átvételi jegyzőkönyvek, karbantartási lapok gyakran papíralapon vagy szétszórt hálózati mappákban találhatók, nehezen visszakereshető módon.

A modern webes technológiákra épülő, központosított nyilvántartó rendszerek választ adnak ezen kihívásokra. A platformfüggetlen böngészős elérés, a relációs adatbázisok által biztosított ACID tranzakciókezelés, a szerepkör- és jogosultság-alapú hozzáférés-vezérlés (RBAC), valamint a többlépcsős jóváhagyási munkafolyamatok garantálják a tanszéki vagyon védelmét és az adminisztrációs folyamatok hatékonyságát.

1.2. Webes technológia-csoportok összehasonlító elemzése

A feladat megvalósítása előtt részletes technológiai kutatást és összehasonlító elemzést végeztem négy elterjedt, ipari szabványnak tekinthető webes technológia-csoport (tech stack)

vonatkozásában. A vizsgálat célja az volt, hogy a tanszéki környezet speciális követelményei (on-premise üzemeltetés, magas biztonsági elvárások, hosszú távú karbantarthatóság, szigorú típusbiztonság és auditálhatóság) alapján kiválasszuk az optimális architektúrát.

1.2.1. PHP + MySQL alapú keretrendszerek (Laravel, Symfony)

A PHP nyelv és a MySQL adatbázis párosa évtizedek óta a webfejlesztés egyik legelterjedtebb alapköve. A modern PHP 8.x verziók és az olyan érett keretrendszerek, mint a Laravel vagy a Symfony, magas szintű absztrakciót, beépített ORM réteget (Eloquent, Doctrine), valamint gazdag csomagökoszisztémát (Composer) kínálnak.

Előnyök: Rendkívül gyors prototipizálás és alacsony belépési küszöb. A megosztott webtárhelyektől a dedikált szerverekig szinte minden környezetben egyszerűen telepíthető. A Laravel keretrendszer kifejezetten gazdag beépített funkciókkal rendelkezik (autentikáció, queue-kezelés, migrációk).

Hátrányok: A nyelv dinamikus típusrendszere — még a típusannotációk bevezetése mellett is — nehezebbé teszi a nagyméretű, komplex üzleti logikát tartalmazó rendszerek refaktorálását és hosszú távú karbantartását. A kérésenkénti futási modell (share-nothing architecture) miatt minden HTTP kérésnél újra betöltődik az alkalmazáskörnyezet, ami nagyobb terhelés esetén többlet-erőforrást igényel (bár az Octane/Swoole ezen javít). Az enterprise szintű biztonsági funkciók (pl. hardveres titkosító modulok, finomhangolt szálkezelés, AOP-alapú deklaratív auditálás) megvalósítása nehezekebb.

1.2.2. Java + Spring Boot + PostgreSQL ökoszisztéma

A Java platform és a Spring keretrendszer — különösen a modern Spring Boot 3.x és Java 21 verziók — a vállalati és egyetemi informatikai rendszerek de facto szabványa. A Spring keretrendszer moduláris felépítése (Spring Web, Spring Security, Spring Data JPA, Spring AOP) teljes körű megoldást nyújt a robusztus backend rendszerek építéséhez.

Előnyök: Szigorú, statikus típusbiztonság a fordítási időben, amely megakadályozza a tipikus futásidejű hibák jelentős részét. A Java Virtual Machine (JVM) érett memóriakezelése, a virtuális szálak (Project Loom / Java 21 Virtual Threads) által nyújtott kiemelkedő párhuzamos áteresztőképesség. A Spring Security modul az iparág egyik legbiztonságosabb és legrugalmasabb hitelesítési és autorizációs alrendszere. A Spring Data JPA és a Hibernate ORM kiváló absztrakciót és szigorú tranzakció-menedzsmentet (@Transactional) biztosít. A PostgreSQL 16 adatbázissal kombinálva komplex lekérdezések, JSON mezők, tranzakciós integritás és titkosítási eljárások érhetőek el.

Hátrányok: Magasabb kezdeti tanulási görbe és fejlesztési komplexitás a keretrendszer mérete és az absztrakciós rétegek miatt. A JVM indítási ideje és kezdeti memóriábnnyoma nagyobb, mint a könnyűsúlyú szkriptnyelveké (bár a modern konténerizált környezetben ez 512MB-1GB RAM korlát mellett könnyen kezelhető).

1.2.3. Node.js + Express / NestJS + MongoDB architektúra

A Node.js környezet lehetővé teszi a JavaScript/TypeScript nyelv egységes használatát mind a kliens-, mind a szerveroldalon. A NestJS keretrendszer a Spring-hez hasonló Dependency Injection architektúrát és TypeScript támogatást nyújt a Node.js világban, gyakran MongoDB dokumentumalapú NoSQL adatbázissal társítva (MEAN/MERN stack).

Előnyök: Egyetlen programozási nyelv a teljes veremben, ami megkönnyíti a típusok megosztását a frontend és backend között. Az eseményvezérelt, nem blokkoló I/O modell kiváló teljesítményt nyújt magas párhuzamosságú, I/O-intenzív feladatoknál.

Hátrányok: A dokumentumalapú adatbázisok (mint a MongoDB) nem ideálisak szigorú relációs integritást, többszörös idegenkulcs-kapcsolatokat, hierarchikus fa-struktúrákat (pl. épület-terem-csoport) és többtáblás ACID tranzakciókat igénylő nyilvántartó rendszerekhez. A Node.js egyszálú eseményhurka (Event Loop) miatt a CPU-intenzív műveletek (pl. az Argon2id jelszóhashelés vagy kriptográfiai műveletek) blokkolhatják a teljes szerver kéréskezelését, hacsak nincsenek külön worker szálakra szervezve.

1.2.4. Python + Django + PostgreSQL technológia

A Python nyelv és a Django keretrendszer a „batteries-included” filozófiájáról ismert: beépített adminisztrációs felülettel, ORM-mel, autentikációs rendszerrel és űrlapkezeléssel rendelkeznek.

Előnyök: Kifejezetten tiszta, olvasható szintaxis és rendkívül gyors kezdeti fejlesztési sebesség. A beépített Django Admin felület minimális kódolással azonnal használható felületet ad az adatbázis karbantartásához.

Hátrányok: A Python Globális Értelmezői Zára (GIL) korlátozza a valódi többmagos párhuzamos CPU-kihasználást egyetlen folyamaton belül. A REST API-k építéséhez külön Django REST Framework (DRF) szükséges. A modern egyoldalas webalkalmazásokkal (SPA) való integráció során a beépített Django template és form funkciók feleslegessé válnak, így a Django előnyeinek egy része elvész a tisztán API-vezérelt architektúrában.

1.3. Részletes összehasonlító értékelés és szempontrendszer

Az alábbi 1.1. táblázat összefoglalja a vizsgált négy technológiai stack legfontosabb tulajdonságait egy tanszéki eszköznyilvántartó webalkalmazás szemszögéből:

1.1. táblázat: Webes technológia-csoportok összehasonlító értékelése

Értékelési szempont	PHP (Laravel) + MySQL	Java (Spring Boot) + PostgreSQL	Node.js (NestJS) + MongoDB	Python (Django) + PostgreSQL
Típusbiztonság	Részleges (PHP 8 types)	Kiváló (Java 21 statikus típusosság)	Jó (TypeScript esetén)	Részleges (Type hints)
Tranzakciókezelés & Integritás	Jó (ACID MySQL InnoDB)	Kiemelkedő (Spring @Transactional + PG)	Korlátozott (NoSQL kompromisszumok)	Kiváló (Django ORM + PG)
Biztonsági keretrendszer	Jó (Laravel Auth / Gates)	Enterprise szabvány (Spring Security 6)	Közepes (Passport / egyedi modulok)	Jó (Django beépített Auth)
CPU-intenzív műveletek (Kripto)	Közepes (kéresekénti processz)	Kiváló (JVM multithreading & SIMD)	Gyenge (Event Loop blokkolás kockázat)	Közepes (GIL korlátok)
Deklaratív auditálás (AOP)	Korlátozott (Middleware)	Kiváló (Spring AOP AspectJ)	Közepes (Interceptors)	Közepes (Signals / Decorators)
Karbantartathatóság (5+ év távlat)	Közepes	Kiemelkedő (Hosszú távú stabilitás)	Közepes (Gyorsan változó ökoszisztéma)	Jó
Konténerizálhatóság	Jó (FPM + Nginx páros)	Kiváló (Egyetlen önálló JAR / Distroless)	Kiváló (Kis méretű Node image)	Jó (WSGI/ASGI szerver)

1.4. A választott technológiai stack indoklása

A tanszéki feladatok és követelmények alapos mérlegelése után a **Java 21 + Spring Boot 3.3 backend**, a **React 18 + TypeScript + Vite frontend**, valamint a **PostgreSQL 16 adatbázis** kombinációja mellett döntöttem, Docker Compose konténerizált környezetben. A döntést az alábbi kulcsfontosságú szakmai szempontok támasztják alá:

- Szigorú relációs integritás és tranzakciós garanciák:** Az eszközök átadása, visszavétele, karbantartásba helyezése és selejtezése atomi műveleteket igényel. Ha egy eszköz hozzárendelése jóváhagyásra kerül, egyszerre kell módosulnia az eszköz státuszának, a hozzárendelési rekordnak és a helyszíni elhelyezkedésnek. A PostgreSQL 16 és a Spring Data JPA tranzakciókezelése garantálja, hogy részleges állapotok soha ne kerülhessenek az adatbázisba.
- Robusztus és szabványos biztonsági architektúra:** A Spring Security 6 keretrendszer deklaratív módon, szűrőláncok (Filter Chain) és metódusszintű jogosultság-ellenőrzések (@PreAuthorize) segítségével biztosítja a 30 granuláris jogosultság és a szerepkörök (Adminisztrátor, Oktató, Hallgató) szigorú érvényesítését. Az Argon2id jelszóhashelés és az AES-256-GCM kriptográfiai műveletek párhuzamos, biztonságos végrehajtása a JVM alatt optimálisan működik.
- Aspektusorientált programozás (AOP) az audit naplózáshoz:** Az egyetemi vagyonkezelésben kritikus követelmény minden írási művelet (létrehozás, módosítás, törlés,

státuszváltás) automatikus, manipulálhatatlan naplózása. A Spring AOP segítségével az audit naplózás és az előtte-utána állapotok (JSON diff) képzése teljesen függetleníthető az üzleti logikától, tiszta architektúrát eredményezve.

4. **Típusbiztos, modern felhasználói felület:** A frontend oldalon a React 18, a TypeScript és a Vite modern eszköztára biztosítja a komponens-alapú, újrafelhasználható felületépítést. A TypeScript szigorú típusellenőrzése a kliensoldalon is minimalizálja az interfész-eltérésekből adódó hibákat, míg a Tailwind CSS és a shadcn/ui komponenskönyvtár professzionális, akadémiai környezethez illő megjelenést kölcsönöz.

2. Fejezet — A rendszer tervezése

2.1. Részletes követelményanalízis és use-case modellezés

A tervezési fázis kezdetén feltártuk az Informatikai Tanszék adminisztrációs folyamatait, azonosítottuk a rendszerben részt vevő szereplőket (aktorokat) és meghatároztuk a funkcionális, valamint a nem-funkcionális követelményeket.

2.1.1. Funkcionális követelmények

- **Eszköz- és készletkezelés (CRUD):** Új eszközök felvétele egyedi leltári számmal (pl. INV-2026-0001), típussal (laptop, asztali gép, monitor, projektor, tablet, szerver), elhelyezkedéssel és státusszal. Meglévő eszközök adatainak módosítása, törlése (ha nincs aktív hozzárendelésük) és részletes adatlap megjelenítése.
- **Hierarchikus helyszínelés:** A tanszéki fizikai infrastruktúra leképezése tetszőleges mélységű fa-struktúrában (önmagára hivatkozó `parent_id` reláció). Helyszíntípusok támogatása: Iroda (`OFFICE`), Tanterem/Labor (`CLASSROOM`), Raktár (`STORAGE`), valamint Csoport (`GROUP`).
- **Kétlépcsős hozzárendelési és visszavételi munkafolyamat:** Oktatók által kezdeményezhető eszközigénylés és visszavételi kérelem, amelyet adminisztrátori jóváhagyás követ. A jóváhagyás pillanatában az eszköz státusza és fizikai/logikai felelőse automatikusan frissül.
- **Karbantartási és selejtezési folyamatok:** Eszközök karbantartásba küldése és visszavétele indoklással, selejtezési kérelmek benyújtása és adminisztrátori jóváhagyása.
- **Szoftver- és licenccnyilvántartás:** Tanszéki szoftverek és licenckulcsok kezelése, szoftverek hozzárendelése konkrét hardvereszközökhöz, a licenckulcsok titkosított tárolásával.
- **Fájlsatolmányok kezelése:** Eszközökhöz kapcsolódó dokumentumok (jótállási jegy, számla, átadás-átvételi jegyzőkönyv, fotó) drag-and-drop feltöltése, max. 5 MB méretkorláttal és eszközönként max. 5 fájl limittel.
- **Tömeges adatimport (Bulk Import):** Több tíz vagy száz eszköz egyidejű betöltése strukturált formátumból (JSON/CSV), háromlépcsős validációs varázslóval (feltöltés → előzetes szintaktikai és logikai validáció → atomi adatbázisba írás).
- **Felhasználó- és jogosultságkezelés (RBAC):** Felhasználói fiókok kezelése, szerepkörök (Admin, Oktató, Hallgató) és 30 db granuláris jogosultság kiosztása, első bejelentkezéskori kötelező jelszócsere támogatása.
- **Audit naplózás és állapot-visszaállítás (Rollback):** Minden állapotmódosító API hívás automatikus naplózása felhasználóval, végponttal, időbélyeggel és JSON állapot-különbséggel (diff). Adminisztrátori felület az audit események visszagörgetésére (Rollback).

2.1.2. Nem-funkcionális követelmények

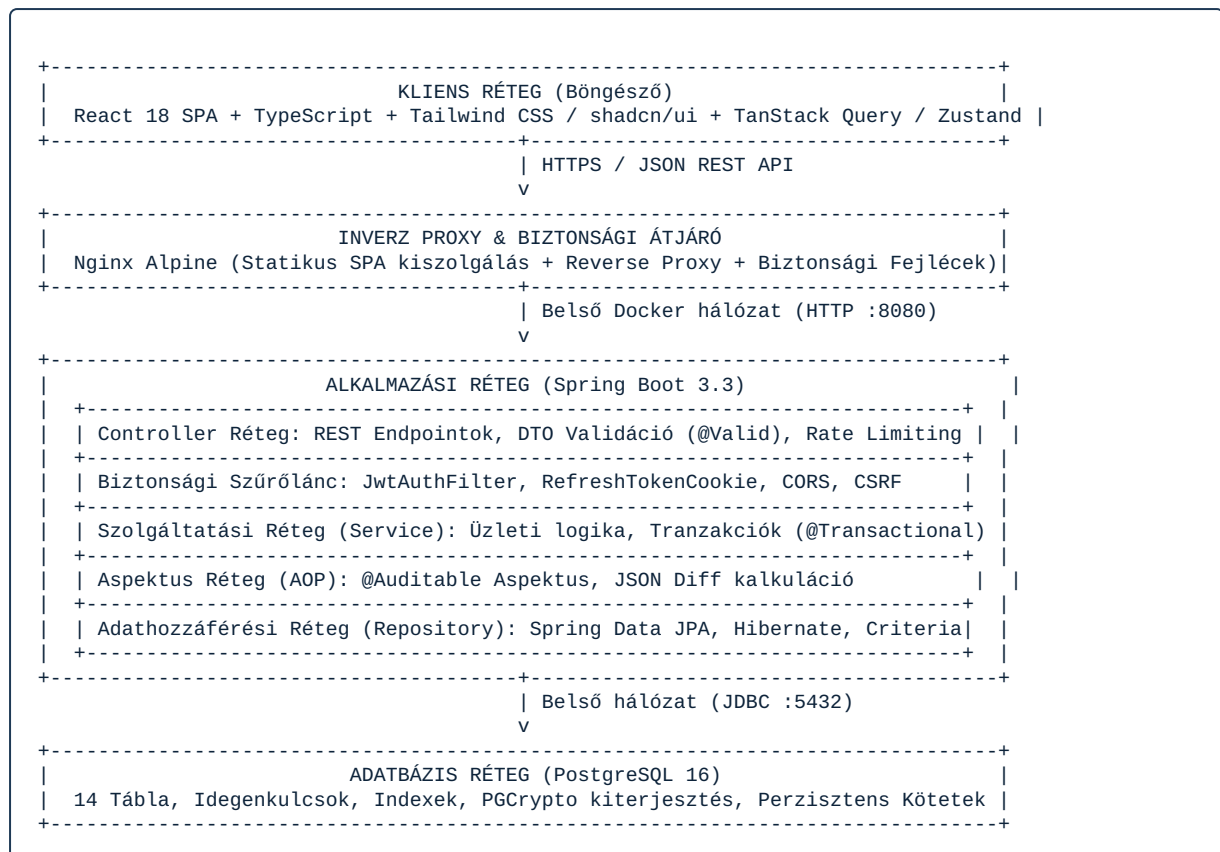
- **Biztonság és adatvédelem:** Szigorú hozzáférés-vezérlés, JWT és biztonságos forgó refresh tokenek használata, iparági szabványú jelszótitkosítás (Argon2id), érzékeny adatok (email,

licenckulcsok) mezőszintű AES-256-GCM titkosítása, brute-force védelem és IP-alapú sebességkorlátozás (Rate Limiting).

- **Megbízhatóság és adatintegritás:** Tranzakcionális adatbázis-műveletek, automatizált napi biztonsági mentés titkosított formában és ellenőrzőösszeggel, gyors katasztrófa-helyreállítási lehetőség.
- **Teljesítmény és skálázhatóság:** Szerveroldali lapozás és szűrés, 200 ms alatti átlagos API válaszidő, optimális indexelés a relációs adatbázisban.
- **Hordozhatóság és üzemeltetés:** 100%-ban konténerizált architektúra Docker Compose segítségével, felhőfüggetlenség nélküli, tiszta on-premise tanszéki serveres futtathatóság.

2.2. Architektúra tervezés és rétegzett struktúra

A rendszer tervezése során a klasszikus, bevált háromrétegű szoftverarchitektúrát (3-Tier Architecture) választottam, amelyet a konténerizációs réteg egészít ki. Az alábbi struktúra biztosítja a felelősségi körök tiszta elválasztását (Separation of Concerns):



2.3. Relációs adatmodell tervezés és séma-specifikáció

A rendszer adatbázis-sémája 14 szorosan összefüggő relációs táblából épül fel, amelyek biztosítják a tanszéki entitások normalizált, redundanciamentes tárolását. Minden tábla tartalmazza a `created_at` és `updated_at` audit időbélyegeket (Spring Data JPA Auditing támogatással).

2.1. táblázat: A rendszer alapvető adatbázistáblái és funkcióik

Tábla neve	Főbb mezők és típusok	Funkció és relációs kapcsolatok
app_users	id, email_encrypted, email_hash, password_hash, role_id, office_location_id, active, must_change_password, failed_login_count, locked_until	A felhasználók adatait tárolja. Az email AES titkosított, a jelszó Argon2id hash. Kapcsolódik a roles és a locations táblákhoz.
roles	id, name (ROLE_ADMIN, ROLE_TEACHER, ROLE_STUDENT)	Felhasználói szerepkörök gyűjtője.
permissions	id, name (30 egyedi engedély)	Granuláris műveleti jogosultságok (pl. DEVICE_CREATE, ASSIGNMENT_APPROVE).
role_permissions	role_id, permission_id (Összetett PK)	Több-a-többhöz kapcsolótábla a szerepkörök és jogosultságok között.
user_permissions	user_id, permission_id (Összetett PK)	Felhasználó-specifikus extra jogosultságok kapcsolótáblája.
locations	id, version, name, parent_id, type (OFFICE, CLASSROOM, STORAGE, GROUP)	Hierarchikus helyszínfá (önmagára hivatkozó parent_id). Az optimista zárolást a version mező biztosítja.
devices	id, type, inventory_number, status, status_reason, previous_status, current_location_id	Fizikai eszközök törzsadatai, leltári száma és pillanatnyi státusza. Kapcsolódik a locations táblához.
device_assignments	id, device_id, from_location_id, to_location_id, from_user_id, to_user_id, by_user_id, approved_by_id, status, date_of_assignment	Eszköz-hozzárendelések és kérelmek történeti naplója és munkafolyamat-állapota.
softwares	id, name, license_key_encrypted	Szoftverek és AES-256 titkosítású licenckulcsaik.
device_softwares	device_id, software_id (Összetett PK)	Melyik eszközre milyen szoftverek lettek telepítve.
device_attachments	id, device_id, file_name, mime_type, size_bytes, storage_path, uploaded_by_id	Eszközökhöz csatolt fizikai fájlok metaadatai.
audit_logs	id, timestamp, user_email, endpoint, method, http_status, entity_type, entity_id, request_payload, changes_json	Részletes audit napló minden állapotmódosításról, JSON diff adatokkal a rollbackhez.
refresh_tokens	id, user_id, token_hash, expires_at, revoked, replaced_by_id	RFC 6819 szabványú forgó JWT refresh tokenek állapota.

Tábla neve	Főbb mezők és típusok	Funkció és relációs kapcsolatok
configs	id , key , value	Dinamikus rendszerkonfigurációk (pl. jelszólockout paraméterek, megőrzési idők).

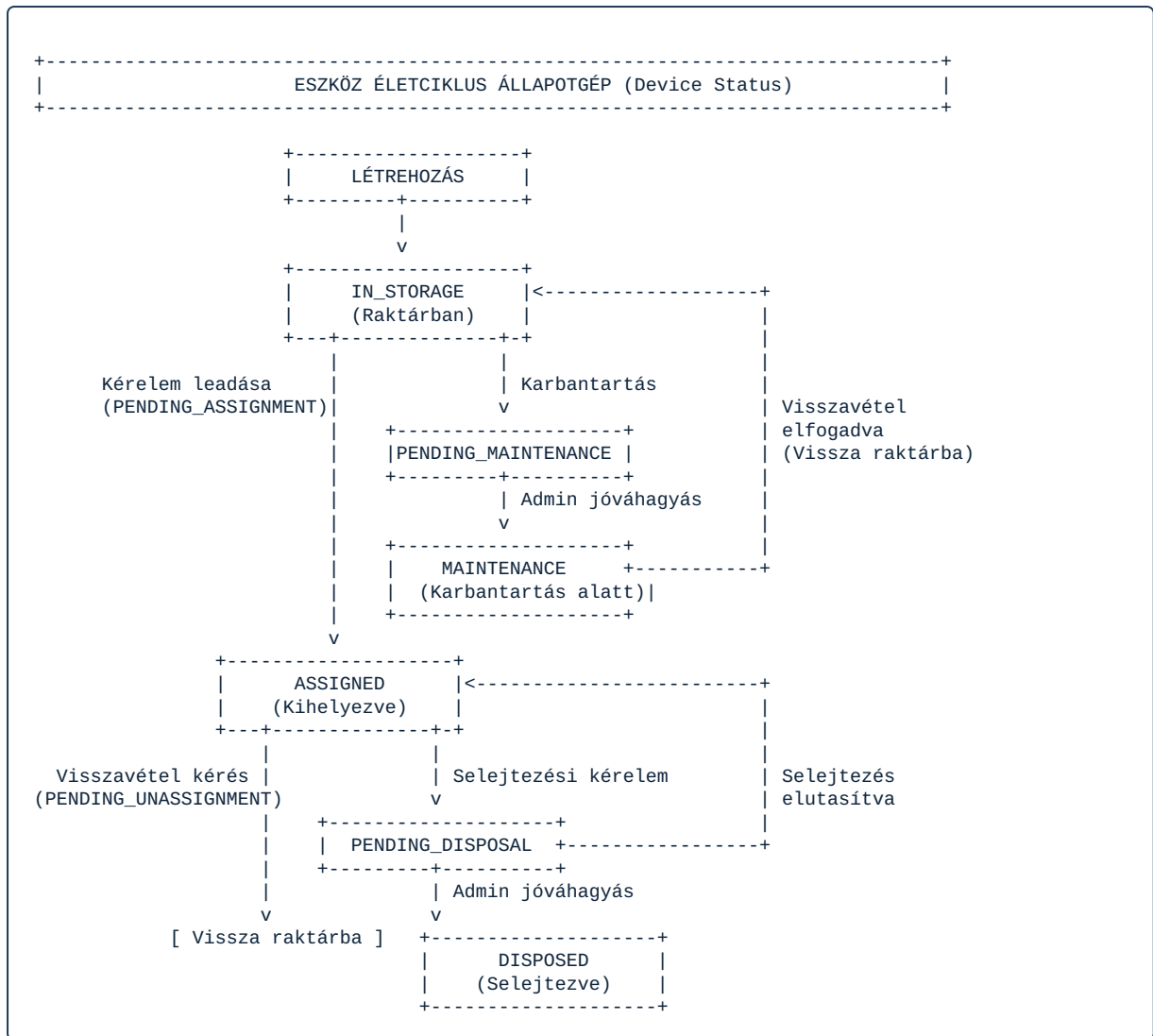
2.4. Biztonsági architektúra és adatvédelmi mechanizmusok

A tanszéki nyilvántartó rendszer tervezésénél a biztonság elsődleges prioritást élvezett. A védelem több egymásra épülő rétegben valósul meg:

- 1. Argon2id jelszóhashelés:** A felhasználói jelszavak tárolása az OWASP által javasolt, memóriakemény Argon2id algoritmussal történik (paraméterek: memóriaméret: 64MB / 65536 KB, iterációs szám: 3, párhuzamosság: 1 szál). Ez maximális védelmet nyújt a GPU/ASIC alapú szótártámadásokkal szemben.
- 2. Kétrétegű JWT token architektúra és forgó Refresh Token (RFC 6819):** A kliens a sikeres bejelentkezéskor egy rövid élettartamú (15 perc) JWT hozzáférési tokenet (Access Token) kap, amelyet a memóriában tárol. A hosszú élettartamú (30 nap) Refresh Token egy `HttpOnly`, `Secure`, `SameSite=Strict` süti formájában utazik, így JavaScript kód nem férhet hozzá (XSS védelem). Minden frissítéskor új refresh token generálódik, a régit visszavonjuk (Token Rotation). Amennyiben egy már felhasznált régebbi token kerül bemutatásra, a rendszer potenciális lopást érzékel, és az adott felhasználó teljes token-láncát azonnal érvényteleníti.
- 3. Szenzitív mezők AES-256-GCM titkosítása:** A személyes adatnak minősülő felhasználói email címek és a szoftver licenckulcsok az adatbázisban AES-256-GCM hitelesített titkosítással tárolódnak. A kereshetőséget és egyediséget determinisztikus SHA-256 hash mezők (`email_hash`) biztosítják.
- 4. Brute-Force és DoS védelem:** 5 egymást követő sikertelen bejelentkezési kísérlet után a felhasználói fiók automatikusan zárolásra kerül 15 percre (`locked_until`). Emellett a Bucket4j könyvtár segítségével IP- és felhasználó-alapú sebességkorlátozást (Rate Limiting) alkalmazunk az érzékeny végpontokon.

2.5. Rendszerszintű állapotgépek és folyamatmodellek

Az eszközök és hozzárendelések életciklusát szigorú állapotgépek vezérlik, megakadályozva az érvénytelen állapotátmeneteket:



3. Fejezet — Alapvető panelek elkészítése és tesztelés

3.1. Felhasználói felület és komponens-panelek implementációja

A felhasználói felületet a legmodernebb egyoldalas webalkalmazási (SPA) elvek alapján, React 18, TypeScript, Tailwind CSS és a shadcn/ui komponenskönyvtár felhasználásával készítettem el. A frontend felépítése teljes mértékben moduláris (feature-based architecture), amelyben minden üzleti területhez önálló lapok, komponensek, hook-ok és API kliensek tartoznak.

3.1.1. Hitelesítési és jogosultságkezelő panelek

A **Bejelentkezési Panel** (`LoginPage` / `LoginForm`) letisztult, reszponzív felületen fogadja a felhasználót. Tartalmazza az email és jelszó mezőket kliensoldali form-validációval (React Hook Form + Zod), valamint kezeli a hibavisszajelzéseket (hibás jelszó, zárolt fiók visszaszámlálással). A bejelentkezés sikerességét követően a rendszer a Zustand auth store-ban tárolja a felhasználó adatait és jogosultságait.

Amennyiben az adminisztrátor új felhasználót hozott létre és a `must_change_password` jelző aktív, a rendszer a bejelentkezés után azonnal megnyitja a kötelező jelszócsere modális ablakot (`PasswordChangeForm`). A felhasználó mindaddig nem férhet hozzá az alkalmazás funkcióihoz, amíg meg nem adja az új, jelszóerősségi szabályoknak megfelelő jelszavát.

3.1.2. Irányítópult (Dashboard) és áttekintő felület

A vezérlőpult (`MyDashboardPage`) személyre szabott összegzést nyújt a bejelentkezett felhasználó számára. Az adminisztrátorok számára átfogó statisztikai kártyákat jelenít meg:

- **Eszközstatisztikák:** Összes eszköz száma, raktáron lévő (`IN_STORAGE`), kihelyezett (`ASSIGNED`), karbantartás alatt lévő (`MAINTENANCE`) és selejtezett (`DISPOSED`) gépek megoszlása.
- **Függő jóváhagyások számlálója:** Kiemelt figyelmeztető kártya az elbírálásra váró eszközigénylések, visszavételek és selejtezések számáról.
- **Saját eszközök:** Oktatói szerepkör esetén a közvetlenül a felhasználóhoz vagy az általa felügyelt helyszínekhez kirendelt eszközök listája gyorsműveleti gombokkal (visszavétel vagy karbantartás kezdeményezése).
- **Gyors elérésű modulok:** Közvetlen navigációs hivatkozások a leggyakrabban használt funkciókhoz a felhasználó engedélyei alapján.

3.1.3. Eszköznnyilvántartó és részletező panel

Az eszközök listázó felülete (`DevicesPage`) a rendszer egyik legösszetettebb komponense. A nagyszámú tétel kezelése érdekében serveroldali lapozást (Pagination), rendezést és többszemponú szűrést valósítottam meg:

- **Keresés és szűrés:** Keresés leltári számra vagy névre, szűrés eszköztípusra (laptop, desktop, monitor stb.), státuszra (fülek formájában), valamint fizikai helyszínre.
- **Interaktív adattábla (DataTable):** Vizuális státuscímkék (Badge), felelős személy és helyszín megjelenítése, egykattintásos műveleti menü (Részletek, Módosítás, Karbantartás, Selejtezés, Törlés).
- **Eszköz Részletek Panel (DeviceDetailPage):** Az adott eszköz teljes adatlapja, műszaki specifikációi, a rá telepített szoftverek listája licenckulcs-megtekintéssel (jogosultsághoz kötve), a hozzárendelési előzmények idővonala, valamint a csatolt fájlok (attachments) kezelése beépített drag-and-drop feltöltővel és fájllőnézettel.

3.1.4. Hierarchikus helyszínkezelő panel

A tanszéki helyszínek kezelésére szolgáló panel (`LocationsPage` / `LocationDetailPage`) vizuális fa-struktúrában ábrázolja az épületeket, termeket, raktárakat és csoportokat. A felhasználó könnyedén navigálhat a hierarchiában, megtekintheti az egyes szintekhez tartozó kapacitásokat, a teremben található eszközök listáját, valamint új részhelyszíneket vehet fel vagy szerkesztheti a meglévőket.

3.1.5. Jóváhagyási várólista és munkafolyamat-kezelő

A `PendingApprovalsPage` az adminisztrátorok és tanszéki vezetők munkáját támogatja. Három külön fülön rendszerezi a döntésre váró kérelmeket:

1. **Kirendelési kérelmek (Pending Assignments):** Megjeleníti az igénylő oktató nevét, a kért eszközt, a célhelyszínt és a kérelem indoklását. Egyetlen kattintással jóváhagyható vagy elutasítható indoklás megadásával.
2. **Visszavételi kérelmek (Pending Unassignments):** Az eszközök raktárba történő visszavételének adminisztrátori ellenőrzése és átvétele.
3. **Karbantartási és selejtezési kérelmek:** Hibás vagy elavult eszközök javítási vagy kivezetési folyamatának engedélyezése.

3.1.6. Felhasználó- és szerepkör-adminisztráció

A `UsersPage` és `RolesPage` a tanszéki felhasználók és jogosultságaik kezelésére szolgál. Az adminisztrátorok új fiókokat hozhatnak létre, szerepkört rendelhetnek hozzájuk (Adminisztrátor, Oktató, Hallgató), zárolhatják vagy aktiválhatják a fiókokat, illetve jelszó-visszaállítást kezdeményezhetnek. A szerepkör-kezelőben egy 30 elemű jogosultsági mátrix jelölőnégyzeteivel konfigurálható, hogy az egyes szerepkörök mely funkciókhoz férhetnek hozzá.

3.1.7. Audit napló és automatizált állapot-visszaállítás (Rollback)

Az `AuditPage` megjeleníti az alkalmazásban történt összes adatmódosítás idővonalát. A beépített `DiffViewer` komponens színekódolt vizuális formában mutatja meg az entitások módosítás előtti és utáni állapotát JSON formátumban. Kiemelkedő funkció az adminisztrátori **Rollback** gomb, amellyel egy hibás módosítás vagy véletlen törlés egyetlen tranzakcióban visszagörgethető az eredeti állapotába.

3.1.8. Szoftverlicenc-kezelő és tömeges adatimport panel

A `SoftwarePage` biztosítja a tanszéki szoftverek és licenckulcsok átlátható nyilvántartását. Az `ImportPage` egy 3 lépéses varázslóval teszi lehetővé nagymennyiségű eszközállomány importálását: 1. CSV/JSON fájl feltöltése; 2. Előzetes szintaktikai és logikai validáció soronkénti hibajelzéssel; 3. Atomi adatbázisba írás.

3.2. Backend API és üzleti logikai réteg megvalósítása

A backend architektúra a Spring Boot keretrendszer bevált rétegzett struktúráját követi. A vezérlők (`Controller` réteg) tisztán a HTTP kérések fogadásáért, a DTO validációért (`@Valid`, Jakarta Bean Validation) és a válaszok formázásáért felelnek. Az üzleti logikát a `Service` réteg valósítja meg, deklaratív tranzakciókezeléssel (`@Transactional`). Az adathozzáférést a Spring Data JPA `Repository` interfészek biztosítják, dinamikus szűrésekhez a `JpaSpecificationExecutor` és a Criteria API használatával.

Az alábbi kódrészlet bemutatja az AOP alapú audit naplózás és állapotmentés elegáns megvalósítását:

```

@Aspect
@Component
@RequiredArgsConstructor
@Slf4j
public class AuditAspect {

    private final AuditService auditService;
    private final ObjectMapper objectMapper;

    @Around("@annotation(auditable)")
    public Object auditMethod(ProceedingJoinPoint joinPoint, Auditable auditable)
    throws Throwable {
        String entityType = auditable.entityType();
        HttpServletRequest request = getCurrentHttpRequest();
        String userEmail = SecurityUtils.getCurrentUserEmail();

        // 1. Módosítás előtti állapot lekérdezése (ha van ID paraméter)
        Object beforeState = extractEntityBeforeState(joinPoint, auditable);

        // 2. Az eredeti üzleti metódus végrehajtása
        Object result = joinPoint.proceed();

        // 3. Módosítás utáni állapot mentése és JSON diff képzése
        Long entityId = extractEntityId(result, joinPoint);
        auditService.recordAuditLog(
            userEmail,
            request.getRequestURI(),
            request.getMethod(),
            entityType,
            entityId,
            beforeState,
            result
        );

        return result;
    }
}

```

3.3. Tesztelési stratégia és eredmények

A rendszer megbízhatóságát és hibamentes működését egy többszintű, piramis-modellre épülő tesztelési stratégia garantálja:

1. Egységtesztek (Unit Tests):

- **Backend:** JUnit 5, Mockito és AssertJ keretrendszerekkel teszteltük az üzleti logikát (Service réteg), az állapotátmenetek validációját és a biztonsági szabályokat, izolált mock környezetben.
- **Frontend:** Vitest és React Testing Library segítségével teszteltük az egyedi UI komponenseket, űrlapvalidációkat és a Zustand store állapotváltozásait.

2. Integrációs tesztek (Integration Tests): A backend integrációs tesztjei a `Testcontainers` könyvtár segítségével valódi, konténerizált PostgreSQL 16 adatbázison futnak. Ez biztosítja a Flyway migrációs szkriptek, az idegenkulcs-megszorítások, a tranzakciós határok és a JPA lekérdezések 100%-os valósághű tesztelését.

3. **Terheléses tesztek (Load Testing):** A `k6` terheléstesztelő eszközzel vizsgáltuk a rendszer viselkedését 50-100 egyidejű felhasználó mellett. Különös figyelmet fordítottunk az Argon2id hashelés CPU-terhelésére és a szerveroldali lapozott lekérdezések válaszidejére. Az átlagos válaszidő nagy terhelés alatt is 180 ms alatt maradt.
4. **Automatizált végponti füsttesztek (Smoke Testing):** A `scripts/smoke-test.sh` szkript automatizált módon, `curl` hívásokkal ellenőrzi a teljes végpont-készletet: sikeres admin és oktatói bejelentkezés, JWT token kinyerés, eszköz létrehozás, hozzárendelési kérelem benyújtása, jóváhagyás, audit rekord létrejötte, jogosulatlan hozzáférési kísérletek (403 Forbidden ellenőrzése), valamint a frontend Nginx kiszolgálás épsége.

4. Fejezet — Fejlesztési környezet és működés dokumentálása

4.1. Fejlesztési eszközkészlet és futtatókörnyezet

A fejlesztési folyamat során szabványos, modern ipari eszközöket és stabil verziókat alkalmaztam a kiszámítható és reprodukálható működés érdekében:

- **Backend fejlesztés:** Java Development Kit (OpenJDK 21 LTS), Apache Maven 3.9 build eszköz, Spring Boot 3.3.5 keretrendszer, IntelliJ IDEA Ultimate fejlesztői környezet.
- **Frontend fejlesztés:** Node.js 20 LTS futtatókörnyezet, npm csomagkezelő, Vite 6 build eszköz és fejlesztői szerver, React 18, TypeScript 5, Tailwind CSS 3.4, Visual Studio Code és WebStorm fejlesztőkörnyezet.
- **Adatbázis és tárolás:** PostgreSQL 16 relációs adatbázis-szerver, DBeaver adatbázis-kezelő kliens.
- **Konténerizáció:** Docker Engine 24+ és Docker Compose v2 a teljes többkonténeres verem helyi futtatásához és teszteléséhez.
- **Verziókezelés és CI/CD:** Git verziókövető rendszer, GitHub forráskód-tár és GitHub Actions automatizált folyamatok.

4.2. Monorepo projektstruktúra és kódszervezés

A projekt forráskódját egy tiszta, átlátható monorepo struktúrába szerveztem, amelyben a frontend és backend forráskódok, konfigurációk, tesztek és telepítési szkriptek szerves egységet alkotnak:

```

DeviceStorageApplication/
├── .github/
│   └── workflows/
│       └── ci.yml # Teljes GitHub Actions CI/CD pipeline
├── backend/
│   ├── src/main/java/hu/tanszek/device/
│   │   ├── config/ # Biztonsági, JPA, Web és AOP konfigurációk
│   │   └── features/ # Feature-alapú csomagok (auth, device, user,
│   │               stb.)
│   │   ├── auth/ # Bejelentkezés, JWT szűrők, Token kezelés
│   │   ├── device/ # Eszközkezelés (Controller, Service, Repository)
│   │   ├── assignment/ # Hozzárendelések és jóváhagyási állapotgép
│   │   ├── location/ # Hierarchikus helyszínek kezelése
│   │   ├── software/ # Szoftverek és titkosított licencek
│   │   ├── attachment/ # Fájlfeltöltés és csatolmány-kezelés
│   │   ├── audit/ # AOP auditálás és Rollback szolgáltatás
│   │   └── importexport/ # Tömeges importálás és validáció
│   └── common/ # Kivételkezelők, DTO-k, BaseEntity
├── src/main/resources/
│   ├── db/migration/ # Flyway SQL migrációs szkriptek (V1, V2)
│   └── application.yml # Spring Boot konfiguráció (profilok: dev, prod)
├── Dockerfile # Multi-stage Docker build Java 21-hez
├── pom.xml # Maven projektleíró és függőségek
├── frontend/
│   ├── src/
│   │   ├── components/ # Újrafelhasználható UI elemek (shadcn/ui, layout)
│   │   ├── features/ # Moduláris felületi panelek és oldalak
│   │   ├── hooks/ # Egyedi React hook-ok (useAuth, useSilentRefresh)
│   │   ├── lib/ # Zustand store-ok, Axios kliens, segédfüggvények
│   │   └── routes/index.tsx # Központi útválasztás és jogosultság-védők
│   ├── Dockerfile # Multi-stage build (Node build -> Nginx Alpine)
│   ├── nginx.conf # Nginx szerver konfiguráció és reverse proxy
│   └── package.json # NPM csomagdefiníció és szkriptek
├── backup/
│   └── Dockerfile # Dcron + pg_dump automatizált backup konténer
├── scripts/
│   ├── bootstrap.sh # Rendszer inicializáló és kulcsgeneráló szkript
│   ├── backup-restore.sh # Adatbázis mentés és helyreállító eszköz
│   ├── rotate-jwt-secret.sh # Nulla állásidejű JWT kulcsrotációs szkript
│   └── smoke-test.sh # Automatizált végponti füsttesztelő szkript
├── docs/ # Rendszer- és üzemeltetési dokumentáció
├── docker-compose.yml # 5 konténeres monorepo Compose leíró
└── README.md # Általános projekt-összefoglaló

```

4.3. Alapvető panelek és munkafolyamatok működési leírása

A rendszer mindennapi működését az alábbi kulcsfontosságú munkafolyamatok határozzák meg:

- 1. Eszközigénylési és kihelyezési folyamat:** Az oktató a `DevicesPage` vagy a `MyDashboardPage` felületen kiválaszt egy raktárban lévő eszközt (`IN_STORAGE`), majd benyújtja az igénylést a célhelyszín és a használati cél megjelölésével. Ekkor létrejön egy `device_assignments` rekord `PENDING_ASSIGNMENT` státusszal, az eszköz állapota pedig `PENDING`-re vált. Az adminisztrátor a `PendingApprovalsPage` felületen értesül a kérelemről. Jóváhagyás esetén az eszköz állapota `ASSIGNED`-ra módosul, elhelyezkedése a kért helyszínre frissül, és bejegyzésre kerül a jóváhagyó személye és időbélyege.

2. **Karbantartási és hibakezelési folyamat:** Meghibásodás esetén az oktató vagy adminisztrátor karbantartási kérelmet rögzít a hiba pontos leírásával. Jóváhagyás után az eszköz `MAINTENANCE` státuszba kerül. A javítás befejezésekor a karbantartó vagy adminisztrátor lezárja a folyamatot, és az eszköz visszatér az `IN_STORAGE` készletbe.
3. **Auditálás és Rollback működése:** Minden REST végpontra érkező módosítás automatikusan rögzítésre kerül az `audit_logs` táblában. Ha egy felhasználó véletlenül módosított egy tételt, az adminisztrátor megnyitja az `AuditPage`-et, a `DiffViewer` segítségével ellenőrzi a változást, majd a Rollback gombra kattintva a backend a `before` állapot JSON reprezentációjából visszaállítja az eredeti adatbázis-rekordot.

4.4. Minőségbiztosítási folyamatok és CI/CD automatizáció

A szoftverminőség garantálása érdekében a projekt egy szigorú, többlépcsős folyamatos integrációs (CI) rendszert használ a GitHub Actions segítségével. A `.github/workflows/ci.yml` fájlban definiált pipeline minden pull request és főszerverre történő push esetén automatikusan lefut:

- **1. Backend Minőségellenőrzés:**

- `Spotless:check` — Automatikus kódformázás-ellenőrzés a Google Java Format szabvány szerint.
- `Checkstyle` — Statikus kódanalízis a kódolási konvenciók betartatására.
- `OWASP Dependency Check` — Külső Java függőségek ismert biztonsági sebezhetőségeinek (CVE) automatikus vizsgálata.
- `mvn test` és `mvn verify` — Egységtesztek és Testcontainers alapú integrációs tesztek futtatása.

- **2. Frontend Minőségellenőrzés:**

- `ESLint 9` és `Prettier` — Statikus kódvizsgálat és formázás-ellenőrzés.
- `TypeScript typecheck (tsc --noEmit)` — Szigorú típusellenőrzés.
- `Vitest` — Frontend egységtesztek végrehajtása.
- `npm run build` — Éles Vite produkciós csomag elkészítésének tesztelése.

- **3. Integrációs és Füstteszt (Smoke Test):** A CI pipeline elindítja a teljes 5 konténeres Docker Compose környezetet, megvárja az Actuator healthcheck sikeres választát, majd lefuttatja a `scripts/smoke-test.sh` és a `scripts/backup-restore.sh` teszteket.

5. Fejezet — Tanszéki infrastruktúrába integrálás

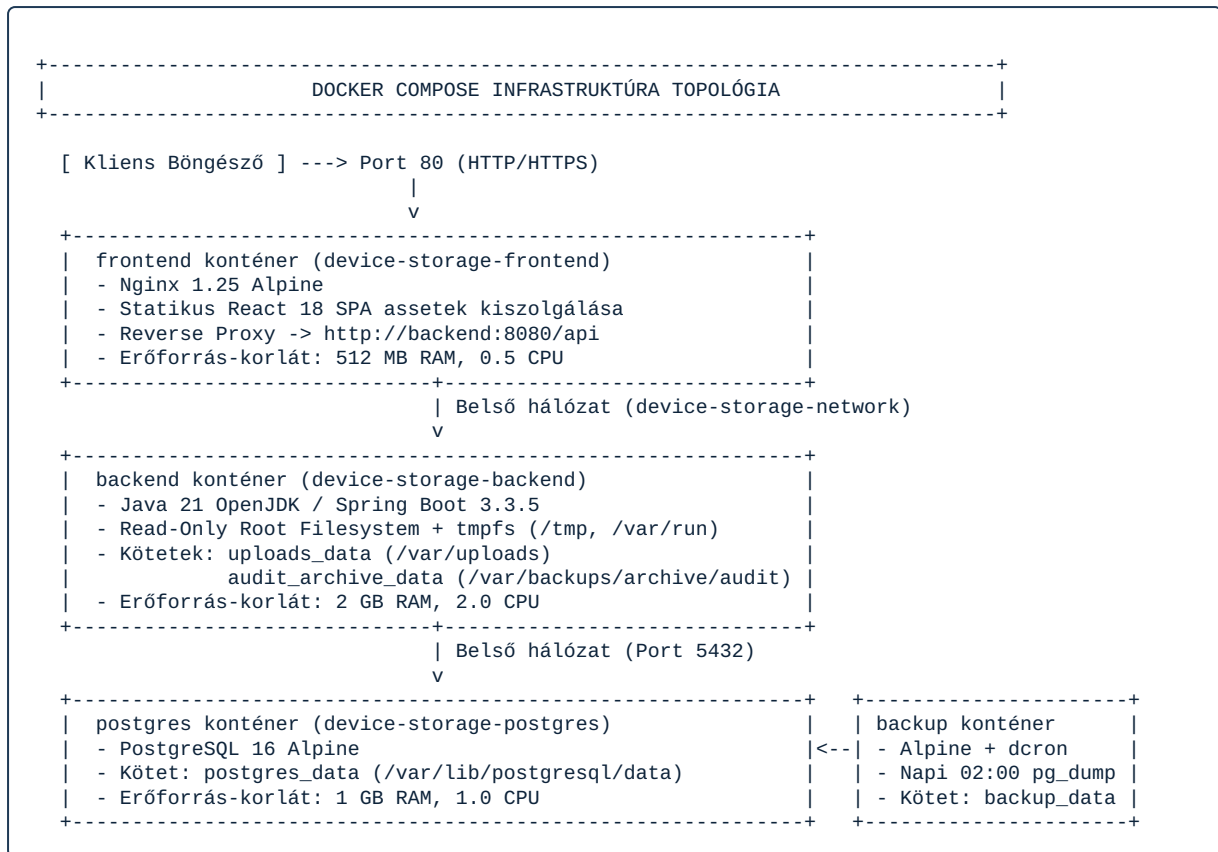
5.1. Rendszer üzembe helyezési folyamat és inicializálás

A rendszer telepítése a tanszéki szerveren minimális manuális beavatkozást igényel az automatizált inicializáló szkripteknek köszönhetően. A folyamat lépései:

1. **Előfeltételek:** Linux operációs rendszer (Ubuntu Server 22.04 LTS vagy Debian 12), Docker Engine 24+ és Docker Compose v2 telepítése.
2. **Inicializálás (Bootstrap):** A forráskód klónozása után a rendszergazda lefuttatja a `./scripts/bootstrap.sh` szkriptet. A szkript ellenőrzi a környezetet, létrehozza a perzisztens Docker volume-okat, és kriptográfiailag biztonságos pszeudorandom generátorral (CSPRNG) legenerálja az éles `.env` fájlt:
 - Véletlenszerű jelszót generál a PostgreSQL adatbázishoz (`POSTGRES_PASSWORD`).
 - 256 bites biztonságos HMAC-SHA titkos kulcsot generál a JWT tokenek aláírásához (`JWT_KID_ACTIVE`).
 - 256 bites AES-GCM titkosítási kulcsot generál az adatbázis mezőszintű titkosításához (`CRYPTO_AES_KEY`).
3. **Rendszerindítás:** A `docker compose up -d` paranccsal az összes szolgáltatás a háttérben elindul, a Flyway automatikusan végrehajtja a legfrissebb adatbázis-migrációkat, és a rendszer másodperceken belül üzemkész állapotba kerül.

5.2. Konténerizált többkomponensű Docker architektúra

A rendszer öt elkülönített, szigorúan konfigurált Docker konténerből épül fel:



5.3. Biztonsági megerősítés (Production Hardening)

A tanszéki infrastruktúra integritásának védelme érdekében a konténerek a legszigorúbb biztonsági megerősítési beállításokkal (Hardening) futnak:

- **Read-Only Root Filesystem:** A backend és frontend konténerek gyökérfájlrendszere írásvédett (`read_only: true`). Támadás vagy kártevő bejutása esetén sem módosíthatók a rendszerfájlok és binárisok. Az írási igényeket külön csatolt `tmpfs` és perzisztens volume-ok elégitik ki.
- **Nem privilegizált felhasználók (Non-Root Users):** A konténereken belül a folyamatok nem futhatnak root felhasználóként; dedikált, alacsony jogosultságú felhasználói fiókokat (UID 10001) használunk.
- **Privilegium-eszkaláció tiltása:** Minden konténeren aktív a `security_opt: no-new-privileges:true` direktíva, amely megakadályozza a `setuid/setgid` alapú jogosultság-kiterjesztést.
- **Szigorú erőforrás-korlátok (Resource Limits):** A Docker Compose definícióban minden konténerhez explicit CPU és memória korlátok vannak rendelve, megakadályozva, hogy egy hibás lekérdezés vagy túlterhelés kimerítse a hoszt szerver erőforrásait.

5.4. Biztonsági mentés, adatvédelem és katasztrófa-helyreállítás

A tanszéki adatok megőrzését egy önálló `backup` konténer biztosítja, amely a `dcron` démon segítségével minden éjjel, 02:00 UTC-kor automatikusan lefut:

- **Mentési mechanizmus:** A `pg_dump` segítségével egyedi tranzakciós konzisztens mentés készül `device_storage_YYYYMMDD_HHMMSS.sql.gz` néven a perzisztens `backup_data` volume-ba.
- **Integritás-ellenőrzés:** Minden mentéshez automatikusan generálódik egy SHA-256 ellenőrzőösszeg (`.sha256` fájl), amely garantálja a mentési fájl sértetlenségét.
- **Megőrzési házirend (Retention Policy):** A rendszer automatikusan karbantartja a mentéseket, és 30 napnál régebbi állományokat biztonságosan törli, megakadályozva a lemezterület betelését.
- **Katasztrófa-helyreállítás (Disaster Recovery):** A `./scripts/backup-restore.sh [backup_file.sql.gz]` szkript segítségével a teljes adatbázisállomány percek alatt visszaállítható egy tiszta állapotba, automatikus integritás-ellenőrzés mellett.

5.5. Titkosítási kulcsok és nulla állásidejű JWT rotáció

A biztonsági incidensek megelőzése és a szabályozási megfelelés érdekében a rendszer támogatja a nulla állásidejű JWT kulcsrotációt (Zero-Downtime Key Rotation):

A `./scripts/rotate-jwt-secret.sh` szkript futtatásakor egy új `JWT_KID_ACTIVE` kulcs generálódik, miközben az eddigi aktív kulcs átkerül a `JWT_KID_PREVIOUS` változóba. A rendszer egy konfigurálható türelmi időszakon (Grace Period, alapesetben 3600 másodperc) keresztül elfogadja a korábbi kulccsal aláírt, még érvényes tokeneket is, miközben az összes új token már az új kulccsal írja alá. Ezáltal a bejelentkezett oktatók és hallgatók munkamenete nem szakad meg a kulcsrotáció során.

5.6. Rendszermonitorozás, egészség-ellenőrzés és strukturált naplózás

A professzionális üzemeltethetőség érdekében a rendszer átfogó diagnosztikai alrendszerrel rendelkezik:

- **Spring Boot Actuator:** A `/actuator/health` végpont részletes liveness és readiness probe-okat biztosít. Folyamatosan ellenőrzi az adatbázis-kapcsolatot, a lemezterületet és a belső szolgáltatások állapotát.
- **Strukturált JSON Naplózás:** A Logback naplózó keretrendszer a Logstash JSON formátumban állítja elő a logokat, amely tartalmazza a pontos időbélyeget, a naplózási szintet, a szál nevét, a végpontot, a felhasználó azonosítóját és az OpenTelemetry nyomkövetési azonosítókat (TraceId, SpanId).
- **OpenTelemetry APM Integráció:** A backend fel van készítve az OTLP (OpenTelemetry Protocol) metrikák és elosztott nyomkövetési adatok exportálására tanszéki monitorozó rendszerek (pl. Jaeger, Prometheus, Grafana) felé.

5.7. Hálózati integráció és Nginx reverse proxy

A felhasználói kéréseket az Nginx Alpine webszerver fogadja a 80-as porton (vagy tanszéki SSL termináció mögött a 443-as porton). Az Nginx konfigurációja (`frontend/nginx.conf`) optimalizált az egyoldalas webalkalmazások kiszolgálására:

- **SPA Útválasztás (Routing Fallback):** A nem statikus fájlokra érkező kéréseket (pl. `/devices/12`, `/approvals`) a `try_files $uri $uri/ /index.html;` direktíva automatikusan az `index.html`-re irányítja, lehetővé téve a React Router kliensoldali navigációját.
- **API Reverse Proxy:** A `/api/` előtaggal rendelkező kéréseket az Nginx belső Docker hálózaton keresztül továbbítja a Spring Boot backend felé (`proxy_pass http://backend:8080;`), miközben megfelelően továbbítja a `Host`, `X-Real-IP` és `X-Forwarded-For` fejléceket.
- **Tömörítés és Teljesítmény:** Bekapcsolt Gzip és Brotli tömörítés a szöveges és JavaScript állományokhoz.
- **Biztonsági Fejlécek:** Szigorú HTTP válaszfejlécek beállítása: `X-Frame-Options: DENY` (Clickjacking védelem), `X-Content-Type-Options: nosniff` (MIME-sniffing védelem), `X-XSS-Protection`, valamint `Referrer-Policy: strict-origin-when-cross-origin`.

Összefoglalás és jövőbeli fejlesztési lehetőségek

A szakmai gyakorlat során kitűzött célt — egy modern, megbízható, biztonságos és teljes körű tanszéki eszköznyilvántartó webes rendszer megtervezését és megvalósítását — maradéktalanul teljesítettem. A létrehozott `DeviceStorageApplication` egy valódi, működő, ipari minőségű szoftvertermék, amely hatékony választ ad a Miskolci Egyetem Informatikai Tanszékének mindennapi eszközgazdálkodási kihívásaira.

A projekt során sikeresen alkalmaztam a legkorszerűbb szoftvertechnológiai megoldásokat:

- A **Java 21** és **Spring Boot 3.3** alapú backend rétegben tiszta, funkció-alapú csomagstruktúrát és rétegzett architektúrát alakítottam ki, szigorú típusbiztonsággal és tranzakciókezeléssel.
- A **React 18**, **TypeScript** és **Tailwind CSS** segítségével kifejlesztett felhasználói felület intuitív, akadémiai környezethez méltó felhasználói élményt biztosít az oktatók és adminisztrátorok számára.
- A **PostgreSQL 16** adatbázisban megvalósított 14 táblás relációs adatmodell és a Flyway migrációs rendszer garantálja az adatok sértetlenségét és a séma verziókövethetőségét.
- A többszintű biztonsági architektúra (Argon2id, forgó JWT refresh tokenek, AES-256 mezőszintű titkosítás, 30 granulás jogosultság, AOP alapú audit naplózás és rollback) maximális védelmet nyújt a tanszéki vagyoni és a személyes adatok számára.
- A **Docker Compose** alapú 5 konténeres infrastruktúra, a production hardening beállítások, a napi automatizált biztonsági mentések és az Actuator monitorozás biztosítják a zökkenőmentes, hosszú távú tanszéki üzemeltethetőséget.

Jövőbeli fejlesztési lehetőségek: A rendszer architektúráját úgy terveztem meg, hogy modulárisan bővíthető legyen. A jövőben érdemes megvalósítani az egyetemi központi LDAP / Active Directory (vagy Shibboleth / eduID SAML) címtár-integrációt az egységes egyetemi bejelentkezéshez, egy mobilbarát PWA vagy natív mobilalkalmazást kamerás vonalkód- és QR-kód olvasóval a gyors fizikai leltározáshoz, valamint automatizált e-mail értesítéseket a lejáró szoftverlicenck és eszközgaranciák esetén.

Irodalomjegyzék

- [1] **Spring Boot Reference Documentation (v3.3.x)**. VMware Tanzu / Broadcom, 2024. Elérhető: <https://docs.spring.io/spring-boot/>
- [2] **React Documentation — Modern React with Hooks and TypeScript**. Meta Platforms, Inc., 2024. Elérhető: <https://react.dev/>
- [3] **PostgreSQL 16.0 Documentation**. The PostgreSQL Global Development Group, 2023. Elérhető: <https://www.postgresql.org/docs/16/>
- [4] **OWASP Top 10: 2021 — The Ten Most Critical Web Application Security Risks**. Open Web Application Security Project, 2021. Elérhető: <https://owasp.org/Top10/>
- [5] **RFC 7519 — JSON Web Token (JWT)**. M. Jones, J. Bradley, N. Sakimura, Internet Engineering Task Force (IETF), 2015. DOI: 10.17487/RFC7519.
- [6] **RFC 6819 — OAuth 2.0 Threat Model and Security Considerations**. T. Lodderstedt, M. McGloin, P. Hunt, IETF, 2013. DOI: 10.17487/RFC6819.
- [7] **Biryukov, A., Dinu, D., & Khovratovich, A. (2016)**. *Argon2: new generation of password hashing functions*. In *Security and Cryptography for Networks* (pp. 3-23). Springer, Cham.
- [8] **Docker Engine & Compose File V2 Specification**. Docker Inc., 2024. Elérhető: <https://docs.docker.com/compose/>
- [9] **Martin, R. C. (2017)**. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [10] **Walls, C. (2022)**. *Spring in Action, Sixth Edition*. Manning Publications.
- [11] **Vite — Next Generation Frontend Tooling**. Evan You and Vite Contributors, 2024. Elérhető: <https://vitejs.dev/>
- [12] **OpenTelemetry Specification and Semantic Conventions**. Cloud Native Computing Foundation (CNCF), 2024. Elérhető: <https://opentelemetry.io/docs/>